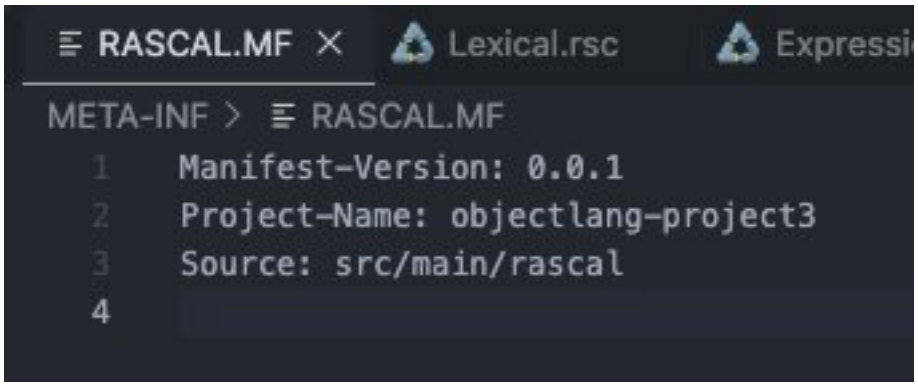
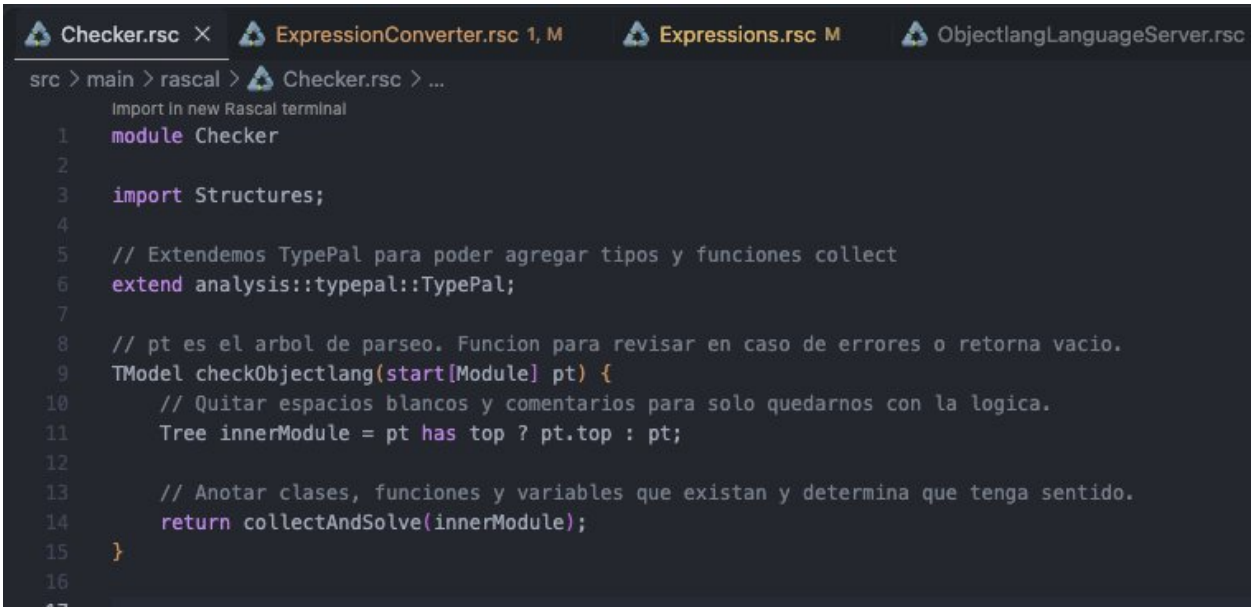
Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

Empezamos configurando el proyecto y cambiando las configuraciones del archivo pom.xml y RASCAL.MF, renombrándolo para tener el nombre indicado de este proyecto. A las dependencias de pom.xml agregamos la dependencia a TypePal y corregimos la extensión que se estaría utilizando para los archivos de salida. Se creo el nuevo archivo de Checker.rsc, en el cual inicialmente se implementó como *stub* para poder avanzar con las demás partes a continuación.





Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548



Esta versión del Checker fue utilizada para poder implementar los demás cambios a nuestro lenguaje y corregir errores de prueba.

Para los tipos primitivos, teniendo en cuenta lo mencionado en el enunciado, agregamos las palabras clave correspondientes y las quitamos del tipo de identificador como regla para que no se puedan utilizar como nombre

```
// Agregar palabras reservadas para los tipos de datos requeridos
keyword TypeKeywords = "Int" | "Bool" | "Char" | "String";
```

```
// No dejamos que los identificadores de tipo sean palabras reservadas
lexical TypeIdentifier =
    ([A-Z][a-zA-Z0-9_]* !>> [a-zA-Z0-9_]) \ TypeKeywords;
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Se agregan explicitamente los tipos que se pueden tener para ser definidos
syntax Type
  = tInt: "Int" !>> [a-zA-Z0-9_]
  | tBool: "Bool" !>> [a-zA-Z0-9_]
  | tChar: "Char" !>> [a-zA-Z0-9_]
  | tString: "String" !>> [a-zA-Z0-9_]
  | simpleType: TypeIdentifier name
  | genericType: TypeIdentifier name "[" {Type ","}+ "]" typeArgs;
```

También tuvimos en cuenta una restricción en los literales y es que, por ejemplo, impedimos que un tipo de usuario se pueda partir en dos. Un ejemplo que nos permite comprender esto mejor es que “Integer” no se llegue a convertir en “Int” y “eger”. Otro aspecto que también dejamos definido fue el literal del Char, para que este solo permita un carácter y sin espacios o cambio de línea y no podemos tener el carácter (‘) como literal.

```
// Literales de un solo caracter
lexical CharLiteral =
    @category="String" [\'] ![\'\n\r] [\'];

start syntax CharTest = CharLiteral;
```

Las llamadas predefinidas print y println reciben exactamente un argumento de cualquier tipo. En el intérprete, print escribe el texto sin salto de línea y println lo escribe con salto de línea; ambas evaluaciones devuelven stringV(text). En el Checker, sin embargo, ambas están declaradas actualmente con tipo de retorno Void. Esta diferencia entre el valor de ejecución y el tipo estático se registra como una limitación pendiente de unificación; no debe afirmarse que ambos componentes devuelven String de manera consistente.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

Para el postfix, agregamos producciones recursivas a la izquierda con memberAccess y call, de tal maneja que podamos ver un oobjeto, buscar su método y ejecutarlo, por ejemplo: puppyFeeder.transform(myPuppy) Con eso podemos hacer llamados a métodos de un objeto de manera mas completa. No solo eso, pero también agregamos el CharLiteral que nos hacia falta en primary.

```
// CharLiteral agregado para soportar literales de un solo caracter
syntax Primary =
    number: IntLiteral
  | identifier: Identifier
  | typeIdentifier: TypeIdentifier
  | charLiteral: CharLiteral
  | booleanLiteral: BoolLiteral
  | StringLiteral
  | grouped: "(" Expr ")"
  ;

syntax Postfix
  = Primary
  | memberAccess: Postfix "." Identifier
  | call: Postfix "(" {Expr ","}* ")"
  ;
```

```
syntax NewExpr
    = newWithArgs: "new" Type "(" {Expr ","}* ")" ClassBody?
    | newWithBody: "new" Type ClassBody
    | newWithoutArgs: "new" Type !>> "("
    ;
```

La gramática ahora hará el trabajo de agrupar lógicamente, entonces el convertidor utilizara pattern matching sobre las nuevas etiquetas, extrae las piezas exactas por medio de los índices y arma los nodos del AST de forma directa.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Acceso a miembros
if (appl(prod(label("memberAccess", _), _, _), _) := current) {
    Expression base = convertExprTree(current.args[0], convertStructural);
    str name = implode(#str, current.args[4]);
    return memberAccess(base, name);
}

//  Llamadas a funciones o métodos
if (appl(prod(label("call", _), _, _), _) := current) {
    Expression base = convertExprTree(current.args[0], convertStructural);
    list[Expression] arguments = convertArgExpressions(current.args[4], convertStructural);
    return call(base, arguments);
}
```

En ObjectlangAST agregamos el charliteral al constructor de Expression Dado que no se maneja un tipo primitivo char, lo ponemos como str asegurando que solo sea de un carácter. Tambien agregamos los constructores explicitos de int, bool, char y string.

```
data Expression
    = number(int literal)
    | addition(Expression left, Expression right)
    | subtraction(Expression left, Expression right)
    | multiplication(Expression left, Expression right)
    | division(Expression left, Expression right)
    | modulo(Expression left, Expression right)
    | stringLiteral(list[StringPart] parts)
    | power(Expression left, Expression right)
    | charLiteral(str character)
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Tipos simples y genéricos
data TypeNode
    = simpleType(str name)
    | genericType(str name, list[TypeNode] typeArgs)
    | tInt()
    | tBool()
    | tChar()
    | tString()
    ;
```

Utilizamos slicing del string para poder convertir la expresión de char

```
if (appl(prod(label("charLiteral", _), _, _), _) := current) {
    str rawText = unparse(current); // Da algo como "'a'"
    return charLiteral(rawText[1..-1]); // Se queda con "a"

}
```

Para el StructuresConverter vamos a agregar los if para determinar las etiquetas de los tipos que se mencionaron anteriormente.

```
// Determinar etiquetas de tipos
if (appl(prod(label("tInt", _), _, _), _) := current) return tInt();
if (appl(prod(label("tBool", _), _, _), _) := current) return tBool();
if (appl(prod(label("tChar", _), _, _), _) := current) return tChar();
if (appl(prod(label("tString", _), _, _), _) := current) return tString();
```

Tambien, para los expression tests, agregamos unas pruebas para el tipo char

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
test bool testCharLiteral() =
    convertExpression("\'a\'") == charLiteral("a");

test bool testCharLiteralEscaped() =
    convertExpression("\'\\n\'") == charLiteral("\\n"); // Si soportas esc
```

Agregamos unas nuevas pruebas a las estructuras también para probar los siguientes elementos:

- Val con primitivo: Determinar que los atributos de una clase pueden usar los nuevos tipos
- Parametro de método: Poder validar los parámetros que se tienen con llamados funciones.
- Parametro de constructor: Prueba que el encabezado de la clase se procesan correctamente.
- Generico con primitivo: Probar que el convertidor sabe entrar a los corchetes de los tipos genéricos y convertir lo de adentro en el tipo establecido.

Para expresiones, probamos los siguientes aspectos:

- Cadenas combinadas de accesos y llamadas
- Llamadas encadenadas

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
/// val con primitivo
test bool testValPrimitive() =
    cleanAST(convertProgram("method main { val x : Int = 5 }")) ==
    moduleProgram([],
        exprStmt(blockExpression(blockNode([
            valDecl("x", [tInt()], number(5))
        ])))
    );

// Parámetro de método con primitivo
test bool testMethodParamPrimitive() =
    cleanAST(convertProgram("object Foo Foo.bar = method (x : Bool) : String method main { }")) ==
    moduleProgram([
        objectDeclaration("Foo", [], [], []),
        methodAssignment("Foo", "bar",
            methodDefinition([methodParameter("x", tBool())], [tString()], [])
        )
    ],
        exprStmt(blockExpression(blockNode([])))
    );

// Parámetro de constructor con primitivo
test bool testConstructorParamPrimitive() =
    cleanAST(convertProgram("object P(val n : Char) method main { }")) ==
    moduleProgram([
        objectDeclaration("P", [], [], [
            constructorParameter("val", "n", tChar())
        ])
    ],
        exprStmt(blockExpression(blockNode([])))
    );

// Genérico con primitivo
test bool testGenericPrimitive() =
    cleanAST(convertProgram("method main { val l : List[String] = x }")) ==
    moduleProgram([],
        exprStmt(blockExpression(blockNode([
            valDecl("l", [genericType("List", [tString()])], identifier("x"))
        ])))
    );
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Cadenas combinadas de accesos y llamadas: a.b(c).d
test bool testChainedMemberAndCall() =
    memberAccess(
        call(
            memberAccess(identifier("a"), "b"),
            [identifier("c")]
        ),
        "d"
    ) == cleanAST(convertExpression("a.b(c).d"));

// Llamadas encadenadas: f(1, 2)(3)
test bool testChainedCalls() =
    call(
        call(identifier("f"), [number(1), number(2)]),
        [number(3)]
    ) == cleanAST(convertExpression("f(1, 2)(3)"));
```

El caso sort("Expr") de convertStatement no le pasaba convertStructuralExpr a convertExprTree, así que un new usado como sentencia sola (el new DogFood del Snippet 3) fallaba. Se corrigió pasando ese argumento y se agregó la prueba testNewAsStatement.

Parser

```
// Punto de entrada único para parsear programas Objectilang.
start[Module] parseObjectlang(loc file)
    = checkNoAmbiguity(parse(#start[Module], file, allowAmbiguity=true, filters={resolveLayoutAmbiguity}));

start[Module] parseObjectlang(str source)
    = checkNoAmbiguity(parse(#start[Module], source, allowAmbiguity=true, filters={resolveLayoutAmbiguity}));
```

Parser.rsc centraliza el análisis sintáctico utilizado en las pruebas y en la ejecución. La variante que recibe loc conserva la ubicación del archivo; la variante que recibe str sirve para analizar texto. El parámetro allowAmbiguity=true y el filtro resolveLayoutAmbiguity permiten descartar alternativas que solo difieren en espacios o saltos de línea. En el

servidor de lenguaje fue necesario configurar explícitamente el mismo filtro en el servicio parsing para evitar un aviso de ambigüedad sobre object Animal. Tras volver a registrar el servidor, VS Code mostró únicamente el error semántico esperado.

```
// Filtro de desambiguación: si todas las alternativas de un nodo ambiguo
// son iguales cuando se ignora el layout, solo difieren en dónde quedó
// un espacio, un salto de línea o un comentario. Se conserva una sola.
Tree resolveLayoutAmbiguity(amb(set[Tree] alternatives)) {
    list[Tree] options = toList(alternatives);
    node reference = withoutLayout(options[0]);
    if (all(Tree other <- options, withoutLayout(other) == reference)) {
        return options[0];
    }
    fail;
}
```

Acá se aplicara cada nodo ambiguo del árbol, se tomara la primera alternativa sin el layout como referencia. Luego, se comparan todas las alternativas con respecto a esa referencia y si todas son iguales, se conserva una. En dado caso que alguna sea diferente, el fail dejara el nodo que es ambiguo intacto. Este problema se presenta porque el layout no va a consumir los saltos de línea por obligación, en cambio debe dejarlos libres para el StatementSeparator. Es por esa razón que object animal seguido de un newline, ese salto puede quedar dentro de objectdef o en el separador de la lista.

```
// Si quedó una ambigüedad que no es de layout, es un error real de la gramática.
start[Module] checkNoAmbiguity(start[Module] tree) {
    if (/a:amb(_) := tree) {
        throw "Ambigüedad real en la gramática: <unparse(a)>";
    }
    return tree;
}
```

Se busca en todas las profundidades del árbol, el nodo antiguo que el filtro mencionado previamente, no pudo resolver. En dado que se encuentre, se lanza un error con el texto ambiguo. El filtro no va a esconder los errores de la gramática. Un caso de ejemplo es new Dog (5), teniendo en cuenta el espacio antes del paréntesis. Ese se puede leer como newWithArgs o como una llamada sobre new Dog. Las dos lecturas van a diferir en estructura, no solamente con el layout, asi que se debe reportar como error.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Copia del árbol con el layout vaciado y sin ubicaciones,
// para comparar solo la estructura.
node withoutLayout(Tree t)
    = unsetRec(visit (t) { case appl(p:prod(layouts(_), _, _), _) => appl(p, []) });
```

El visit va a reocrrer el árbol y reemplaza cada nodo de layout por uno que este vacio, mientras que el unsetRec borra las ubicaciones. Si no ponemos el unsetRec, entonces las dos alternativas iguales se llegarían a ver distinto, debido a que los nodos empiezan o terminan en posiciones diferentes.

```
// String completo
lexical StringLiteral =
    @category="string" stringLiteral: "\"" StringItems "\"";
```

En Expressions antes teníamos el StringLiteral como syntax, y ahora lo tenemos como lexical. Esto se debe a que en la producción de Syntax, Rascal nos inserta layout entre los símbolos, incluso entre comillas y lo que se contiene. Por eso, en uno de los ejemplos, “ Cats Meow “, los espacios que se encuentran al inicio se perdían y los del final quedaban ambiguos. En el caso del tipo String, los espacios son contenido y no separadores, y la producción en Rascal de lexical no instera el layout. Las interpolaciones ${ expr } se siguem aceptando porque StringInterpolation sigue siendo de syntax.

```
// Strings simples e interpolados
if (appl(prod(label("stringLiteral", _), _, _), _) := current) {
    list[StringPart] parts = [];

    Tree stringItems = current.args[1];

    for (item <- stringItems.args[0].args) {
        parts += convertStringPart(item, convertStructural);
    }

    return stringLiteral(parts);
}
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

En el ExpressionConverter, la línea antes tenia current.args[2] y ahora en cambio tiene 1. Entonces como no hay layout, va a desaparecer el hijo de este que se encontraba entre la comilla y lo que se contiene, y StringItems entonces pasara de la posición 12 a la 1.

```
192         if (appl(prod(sort("Expr"), _, _), _) := current) {
193             return exprStmt(convertExprTree(current, convertStructuralExpr));
194         }
```

En el StructuresConverter, antes teníamos return exprStmt(convertExprtTree(current));. Esto convertia una expresión usada como sentencia. Con el solo argumento, usaría por defecto unsupportedStructuralExpr, que lanzara en error en el momento que encuentre un new. Ahora con el convertStructuralExpr, la conversión de new se va a delegar a convertNew. Se deja este caso igual que los otros tipos de sentencia del mismo método que ya pasaban ese argumento.

```
89    test bool testNewAsStatement() =
90        moduleProgram(
91            [],
92            exprStmt(blockExpression(blockNode([
93                exprStmt(newObject(simpleType("Dog"), [], []))
94            ])))
95        ) == cleanAST(convertProgram("method main { new Dog }"));
96
```

Para el StructuresTests, agregamos el caso de new, donde se va a reproducir el caso de new DogFood del snippet 3 del primer enunciado, donde new es usado como una sentencia sola.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
19  void runObjectlang(loc fileLoc) {
20      try {
21          // Parseo directo desde el loc (conserva línea/columna)
22          start[Module] pt = parseObjectlang(fileLoc);
23
24          // Chequeo
25          list[Message] errors = [m | m <- checkObjectlang(pt).messages, m is error];
26
27          if (size(errors) > 0) {
28              println("Se encontraron errores estáticos. Ejecución detenida:");
29              for (Message e <- errors) {
30                  println("- <e.msg> en <e.at>");
31              }
32              return;
33          }
34
35          // Conversión al AST usando la raíz desempacada
36          Program ast = convertModuleTree(pt.top);
37
38          // Ejecución
39          runProgram(ast);
40
41      } catch ParseError(loc l): {
42          println("Error de sintaxis en <l.file> (Línea <l.begin.line>, Columna <l.begin.column>)");
43      } catch str problema: {
44          println("Error: <problema>");
45      }
46  }
47
```

Con el loc, los errores me permiten reportar el archivo, línea y columna. Estos se van a filtrar con el m is error, debido a que TypePal retorna mensajes que no deberían detener la ejecución del programa. En dado caso que haya errores de tipo, el programa no se va a ejecutar. Con esto, podemos garantizar que debe darnos un sistema de tipos. El pt.top va a quitar el start y se reutiliza el árbol ya parseado. Luego se llama al interprete y se lanzan excepciones en dado caso haya algún error.

Interpreter

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
8   // Los valores de ejecución
9   data Value
10    = intV(int n)
11    | boolV(bool b)
12    | charV(str c)
13    | stringV(str s)
14    | rangeV(int from, int to)
15    | objectV(str typeName, map[str, Value] fields, map[str, OverrideDefinition] overrides)
16    ;
17          module Interpreter
18  // El entorno
19  alias Env = map[str, Value];
20
21  // Tablas globales del programa: definiciones de objetos y métodos asignados
22  map[str, Declaration] objectTable = ();
23  map[str, map[str, Declaration]] methodTable = ();
```

Para el interpreter, iniciamos con un constructor para cada tipo de valor que puede tenerse al ejecutar el programa. Objectv guarda el nombre del tipo, los campos y el override del objeto en cuestión. El entorno va a asociar cada nombre de variable a su valor y las definiciones de objetos por su nombre y los métodos asignados por el tipo y el nombre del método.

```
25  void runProgram(moduleProgram(list[Declaration] decls, Statement main)) {
26      objectTable = ();
27      methodTable = ();
28      for (Declaration d <- decls) register(d);
29      exec(main, ());
30  }
31
32  void register(decl:objectDeclaration(str name, _, _, _)) {
33      objectTable[name] = decl;
34  }
35
36  void register(methodAssignment(str target, str name, Declaration def)) {
37      methodTable[target] = (methodTable[target] ? ()) + (name : def);
38  }
```

Luego, vamos a limpiar las tablas par que la ejecución actual no pueda usar los datos de la ejecución anterior. Se registran todas las declaraciones, antes de ejecutar el main. De esta forma, un objeto o método puede usarse aunque este declarado mas abajo en el archivo. Despues se ejecutara el main en un entorno completamente vacio. El operador de (?) devuelve los métodos que tienen ya su tipo, o un mapa vacio si es el primero, y agregara el nuevo.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// toText: Representación textual
str toText(intV(int n)) = "<n>";
str toText(boolV(bool b)) = "<b>";
str toText(charV(str c)) = c;
str toText(stringV(str s)) = s;
str toText(rangeV(int a, int b)) = "<a> -\> <b>";
str toText(objectV(str t, _, _)) = t;

// print y println
Value eval(call(identifier("print"), [Expression arg]), Env env) {
    str text = toText(eval(arg, env));
    print(text);
    return stringV(text);
}

Value eval(call(identifier("println"), [Expression arg]), Env env) {
    str text = toText(eval(arg, env));
    println(text);
    return stringV(text);
}
```

El objeto se va a imprimir como el nombre de su tipo, por ejemplo en el caso del snippet 3 donde tenemos impreso DogFood. El toText se declara 6 veces, una para cada constructor generado en value previamente. Cada versión tiene un patron especificado en su parámetro, y luego Rascal ejecutara lo que coincide con el valor que recibe. Entonces por ejemplo para intV(14) se devuelve “14”, para charV(“z”) da “z”, para range(2, 4) da “2 -> 4” y asi en adelante. Luego sigue el eval de print y println. Para print, como tal no se tiene un nodo propio en la gramática ni en el AST. Entonces cuando el programa tiene este, el parser reconoce con al producción de call de postfix y el convertidor lo transforma en call(identifier(“print”), [identifier(“x”)]), lo cual se prueba en testPrintX y el interprete recibe ese nodo call y busca la versión de eval que coincide.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
60    // Despacho por patrones
61    Value eval(number(int n), Env env) = intV(n);
62    Value eval(booleanLiteral(bool b), Env env) = boolV(b);
63    Value eval(charLiteral(str c), Env env) = charV(c);
64
65    // Variables y Errores de ejecución
66    Value eval(identifier(str name), Env env) {
67        if (name in env) return env[name];
68        throw "Variable no definida: <name>";
69    }
70
```

Ahora, se convierten un literal del AST en su valor de ejecución, entonces el proceso mencionado anteriormente pero al reves. 5 se vuelve en intV(5), z se convierte en charV(z) y asi en adelante. Si el nombre se encuentra en el entorno, se devuelve su valor y si no, entonces hay un error de ejecución.

```
74  // Aritméticos
75  Value eval(addition(Expression l, Expression r), Env env) = intV(x + y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
76  Value eval(subtraction(Expression l, Expression r), Env env) = intV(x - y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
77  Value eval(multiplication(Expression l, Expression r), Env env) = intV(x * y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
78
79  Value eval(division(Expression l, Expression r), Env env) {
80      if (intV(x) := eval(l, env), intV(y) := eval(r, env)) {
81          if (y == 0) throw "División por cero";
82          return intV(x / y);
83      }
84      throw "La división espera dos enteros";
85  }
86
87  Value eval(modulo(Expression l, Expression r), Env env) {
88      if (intV(x) := eval(l, env), intV(y) := eval(r, env)) {
89          if (y == 0) throw "Módulo por cero";
90          return intV(x % y);
91      }
92      throw "El módulo espera dos enteros";
93  }
94
95  Value eval(power(Expression l, Expression r), Env env) {
96      if (intV(base) := eval(l, env), intV(exp) := eval(r, env)) {
97          if (exp < 0) throw "El exponente no puede ser negativo";
98          return intV(intPower(base, exp));
99      }
100     throw "La potencia espera dos enteros";
101 }
102
103 int intPower(int base, int exp) = exp == 0 ? 1 : base * intPower(base, exp - 1);
104
105 Value eval(unaryMinus(Expression e), Env env) = intV(-x) when intV(x) := eval(e, env);
106
```

El when nos va a permitir que la función solamente se aplique en dado caso que ambos operandos que se están manejando, sean enteros en caso de multiplicación, resta y suma. Para la división y el modulo, el if se usa para verificar los tipos pero también el

divisor para que no se pueda dividir entre 0. Dado que Rascal no posee como tal un operador de potencia entera, el intPower lo va a calcular por medio de recursión. Por temas de facilidad, no aceptamos potencias negativas. El unaryMinus cambia el signo de un entero.

```
// Comparaciones
Value eval(lessThan(Expression l, Expression r), Env env) = boolV(x < y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
Value eval(greaterThan(Expression l, Expression r), Env env) = boolV(x > y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
Value eval(lessOrEqual(Expression l, Expression r), Env env) = boolV(x <= y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
Value eval(greaterOrEqual(Expression l, Expression r), Env env) = boolV(x >= y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
Value eval(equal(Expression l, Expression r), Env env) = boolV(eval(l, env) == eval(r, env));
Value eval(notEqual(Expression l, Expression r), Env env) = boolV(eval(l, env) != eval(r, env));
```

Para los operadores <, >, <= y >= solo se van a aceptar enteros, mientras que para == y <> se comparan dos valores completos y de cualquier tipo.

```
115    // Lógicos (con cortocircuito)
116    Value eval(logicalAnd(Expression l, Expression r), Env env) {
117        Value left = eval(l, env);
118        if (boolV(false) := left) return boolV(false);
119        if (boolV(true) := left, boolV(b) := eval(r, env)) return boolV(b);
120        throw "\'and\' espera dos booleanos";
121    }
122
123    Value eval(logicalOr(Expression l, Expression r), Env env) {
124        Value left = eval(l, env);
125        if (boolV(true) := left) return boolV(true);
126        if (boolV(false) := left, boolV(b) := eval(r, env)) return boolV(b);
127        throw "\'or\' espera dos booleanos";
128    }
129
130    Value eval(unaryNeg(Expression e), Env env) = boolV(!b) when boolV(b) := eval(e, env);
131
132    Value eval(grouped(Expression inner), Env env) = eval(inner, env);
```

Para el caso del and, solo se evalua el lado izquierdo y se guarda el resultado. Si el lado izquierdo llega a ser false, el resultado será ese sin importar lo que se tenga en el lado derecho. Como el lado derecho nunca se retorna, entonces esto es a lo que hace referencia el comentario de cortocircuito. En dado caso de que el resultado sea verdadero, toca evaluar el derecho. Se evalua y si es booleano, ese será el resultado. Si alguno no cumple con el tipo, se da error. Para el or es lo mismo pero con el true, si el izquierdo es verdad entonces no se evalua el derecho. Al final, el primer Value eval va a invertir el booleano. El when va a exigir un valor que sea booleano y en dado caso de que no sea y no coincide ningún caso, salta al error. Solo evaluaremos lo de adentro del paréntesis, ya que el agrupamiento fue resuelto por el parser cuando se hizo el árbol, asi que en ejecución los paréntesis no hacen nada más.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
134    // Objetos
135    str typeNameOf(simpleType(str n)) = n;
136    str typeNameOf(genericType(str n, _)) = n;   // los genéricos se borran en ejecución
137    default str typeNameOf(TypeNode t) { throw "No se pueden crear objetos de un tipo primitivo"; }
```

Se hace un borrado de los genéricos, ya que en ejecución List[Dog] solo será List. Esto se debe a que los argumentos genéricos serán información estatica, es decir que el checker verifica que correspondan los tipos antes de ejecutarse.

```
139    Value eval(newObject(TypeNode t, list[Expression] args, list[OverrideDefinition] overrides), Env env) {
140        str name = typeNameOf(t);
141        if (name in objectTable, objectDeclaration(_, _, _, list[ConstructorParameter] params) := objectTable[name]) {
142            list[Value] argValues = [eval(a, env) | a <- args];
143            if (size(params) != size(argValues)) {
144                throw "<name> espera <size(params)> argumentos y recibió <size(argValues)>";
145            }
146            map[str, Value] fields = (params[i].name : argValues[i] | i <- [0..size(params)]);
147            map[str, OverrideDefinition] ovs = (ov.name : ov | ov <- overrides);
148            return objectV(name, fields, ovs);
149        }
150        throw "Objeto no definido: <name>";
151    }
```

Entonces, se busca la definición del objeto, se verifica que la cantidad de argumentos dado coincida con los parámetros que se establecieron en el constructor y que cada parámetro del constructor se convierta en un campo con el valor del argumento al cual le corresponde. Finalmente, se guarda los override escritos al crear el objeto y se indexan por el nombre.

```
// Acceso a un campo: obj.campo
Value eval(memberAccess(Expression target, str field), Env env) {
    if (objectV(str t, map[str, Value] fields, _) := eval(target, env)) {
        if (field in fields) return fields[field];
        throw "El objeto <t> no tiene el campo <field>";
    }
    throw "Solo se puede acceder a campos de un objeto: <field>";
}
```

Para el acceso a miembros de un objeto, se evalua lo que esta antes del punto y se verifica que sea un objeto. Si lo es, se guarda el nombre de su tipo en t y sus campos en fields. Luego se busca el campo, y en dado caso que exista en el mapa de campos, entonces se devuelve el valor. Si el objeto existe, pero no tiene campo, lanza un error que nombra el tipo y el campo.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// Llamada a un método: obj.metodo(args)
Value eval(call(memberAccess(Expression target, str m), list[Expression] args), Env env) {
    if (objectV(str t, map[str, Value] fields, map[str, OverrideDefinition] overrides) := eval(target, env)) {
        list[Value] argValues = [eval(a, env) | a <- args];
        if (m in overrides, overrideDefinition(_, list[MethodParameter] ps, _, BlockNode body) := overrides[m]) {
            return invoke(t, m, ps, body, fields, argValues);
        }
        if (methodDefinition(list[MethodParameter] ps, _, list[BlockNode] bodies) := findMethod(t, m)) {
            if (bodies == []) throw "El método <t>.<m> es abstracto y no tiene implementación";
            return invoke(t, m, ps, bodies[0], fields, argValues);
        }
    }
    throw "Solo se pueden llamar métodos sobre objetos: <m>";
}
```

Para este caso, se evalua el objeto que recibe la llamada. Se exige que sea un objeto y se guardan tres cosas: el tipo, campos y sus overrides. Se evaluan los argumentos de izquierda a derecha y en el entorno de quien lo llama. Luego, se busca un override en el propio objeto. Si al crearlo, se le dio un override con ese nombre, se sacan sus parámetros y su cuerpo, y se ejecuta con el invoke. Si no hay override, entocnes busca el método del tipo con el findMethod y esa función buscara primero en el tipo del objeto. Si no lo encuentra, sube al supertipo de extends y repite, hasta encontrarlo. Si al final no encontró nada, lanza la excepción. Si el método se encontró pero no tiene cuerpo, es abstracto y no se ejecute, entonces es error. Si tiene cuerpo, ejecuta el invoke y si lo que esta antes del punto no es objeto, se hace error.

```
Declaration findMethod(str typeName, str m) {
    if (typeName in methodTable, m in methodTable[typeName]) return methodTable[typeName][m];
    if (typeName in objectTable, objectDeclaration(_, _, [TypeNode sup], _) := objectTable[typeName]) {
        return findMethod(typeNameOf(sup), m);
    }
    throw "Método no encontrado: <typeName>.<m>";
}
```

Esta es la función mencionada anteriormente. Se va a buscar el método con el tipo de objeto y si no está, sube al supertipo declarado en extends y repite la búsqueda hasta encontrar algo o no.

```
// Dentro del método se ven los campos del objeto y los parámetros.
Value invoke(str t, str m, list[MethodParameter] ps, BlockNode body, map[str, Value] fields, list[Value] argValues) {
    if (size(ps) != size(argValues)) {
        throw "<t>.<m> espera <size(ps)> argumentos y recibió <size(argValues)>";
    }
    Env methodEnv = fields + (ps[i].name : argValues[i] | i <- [0..size(ps)]);
    return eval(blockExpression(body), methodEnv);
}
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

El entorno del método se forma con los campos del objeto y sus parámetros. En este caso, los campos se ven directamente por nombre. El método retorna el valor de su bloque y el de su última sentencia.

```
289     // Casos por defecto para capturar errores de AST
290     default Value eval(Expression e, Env env) {
291         throw "Falta implementar evaluación para: <e>";
292     }
293     default tuple[Value, Env] exec(Statement s, Env env) {
294         throw "Falta implementar ejecución para: <s>";
295     }
```

El default hace que estos dos casos que se presentan solo se usen cuando ninguno de los patrones llegue a coincidir. De esta manera, cualquier producción no soportada produce un error claro en lugar de una falla interna.

```
207
208   // Strings y su interpolación
209   Value eval(stringLiteral(list[StringPart] parts), Env env) {
210       str result = "";
211       for (p <- parts) {
212           if (textPart(str text) := p) result += text;
213           else if (escapePart(str esc) := p) {
214               switch(esc) {
215                   case "\\n": result += "\n";
216                   case "\\t": result += "\t";
217                   case "\\r": result += "\r";
218                   case "\\\"": result += "\"";
219                   case "\\$": result += "$";
220                   case "\\\\": result += "\\";
221                   default: result += esc;
222               }
223           }
224           else if (identifierInterpolation(str name) := p) result += toText(eval(identifier(name), env));
225           else if (expressionInterpolation(Expression e) := p) result += toText(eval(e, env));
226       }
227       return stringV(result);
228   }
```

Para el string e interpolación, empezamos con un texto vacio y recorremos los pedazos en orden. El texto literal se agrega tal cual, los espacios al inicio y del final se convervan gracias a StringLiteral que se volvió lexico. En el AST, un esc es guardado como dos caracteres, la barra y una letra. Poir ejemplo \n es una barra seguida de una n. El switch nos permitirá traducir el carácter real que lo representa.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
// El bloque y el main
Value eval(blockExpression(blockNode(list[Statement] stmts)), Env env) {
    Value lastVal = intV(0); // Por si el bloque está vacío
    Env currentEnv = env;
    for (s <- stmts) {
        tuple[Value, Env] res = exec(s, currentEnv);
        lastVal = res[0];
        currentEnv = res[1]; // Actualizamos el entorno para la siguiente sentencia
    }
    return lastVal;
}

// Ejecución de Sentencias
tuple[Value, Env] exec(valDecl(str name, _, Expression expr), Env env) {
    Value v = eval(expr, env);
    return <v, env + (name : v)>; // Se agrega el nombre al entorno

}

tuple[Value, Env] exec(varDecl(str name, _, Expression expr), Env env) {
    Value v = eval(expr, env);
    return <v, env + (name : v)>;
}

// Asignación
tuple[Value, Env] exec(assignmentStmt(identifier(str name), Expression expr), Env env) {
    if (name notin env) thro: "Variable no definida: <name>";
    Value v = eval(expr, en  map[str, Value]
    return <v, env + (name : v)>;
}
```

El entorno que va dejando cada sentencia es la que recibe la siguiente. De esa forma, un val declarado en una línea es visible en las líneas que siguen. El valor de un bloque será el de su ultima sentencia, es decir que se retorna un método sin necesidad de return. Por ejemplo, el new DogFood del final del override del snippet 3. Luego val y var agregan el nombre al entorno. Y finalmente solo se podrá asignar a una variable que ya existe.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

```
194  // Rangos (inclusivos)
195  Value eval(range(Expression l, Expression r), Env env) = rangeV(x, y) when intV(x) := eval(l, env), intV(y) := eval(r, env);
196
197  Value eval(inExpression(Expression e, Expression c), Env env) {
198      if (intV(x) := eval(e, env), rangeV(a, b) := eval(c, env)) return boolV(a <= x && x <= b);
199      throw "\'in\' espera un entero y un rango";
200  }
201
202  Value eval(conditional(Expression cond, Expression thn, Expression els), Env env) {
203      if (boolV(true) := eval(cond, env)) return eval(thn, env);
204      return eval(els, env);
205  }
206
```

```
261  // Ciclo for sobre rangos
262  // El cuerpo puede modificar variables de afuera, pero lo que declara
263  // adentro (incluida la variable del ciclo) no sale del ciclo.
264  tuple[Value, Env] exec(forStmt(forBinding(idPattern(str name), _, Expression expr), Statement body), Env env) {
265      if (rangeV(int a, int b) := eval(expr, env)) {
266          Env current = env;
267          list[int] values = a <= b ? [a..b + 1] : [];
268          for (int i <- values) {
269              tuple[Value, Env] res = exec(body, current + (name : intV(i)));
270              Env updated = (k : res[1][k] | k <- current, k != name);
271              if (name in current) updated[name] = current[name];
272              current = updated;
273          }
274          return <intV(0), current>;
275      }
276      throw "El for espera un rango";
277  }
278                      Statement, map[str, Value]
279  tuple[Value, Env] exec(forStmt(forBinding(tuplePattern(_, _), _, _), _), Env env) {
280      throw "Patrón de tupla no soportado en ejecución";
281  }
```

Primeramente, el a -> b se va a evaluar como un rango. Luego, el x in r va a comprobar si x esta dentro del rango, incluyendo los extremos de este. Entonces [a…b+1] va a producir a, a+1,…,b porque en Rascal el límite superior del intervalo no se va a incluir. Si a > b entonces la lista queda vacia y el ciclo no se itera. El cuerpo se va a ejecutar con la variable del ciclo ligada al valor actual y después de cada iteración, se conservan solo las variables que ya existían antes del ciclo, con sus valores nuevos. Si afuera ya existía una variable con el mismo nombre que la del ciclo, se restaura el valor original. El patron de tupla se reportara como error, porque3 el lenguaje no tiene valores tupla.

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

Comprobación estática de tipos

Después de tener funcionando la gramática, los convertidores y el intérprete, continuamos con el Checker.rsc. En esta parte necesitábamos revisar los tipos antes de ejecutar el programa. La idea era que si, por ejemplo, se declaraba una variable de tipo Int y se le asignaba un String, el programa reportara el error con su ubicación en lugar de llegar hasta la ejecución.

Integración con TypePal

Para esto usamos TypePal. La función checkObjectlang recibe el árbol start[Module] que viene del parser, quita el nivel start cuando es necesario y llama a collectAndSolve. En la etapa de recolección registramos primero print y println, después los objetos y sus scopes, los parámetros genéricos, los parámetros de constructores, las asignaciones de métodos y finalmente el main. Fue importante separar los roles de los identificadores, porque un nombre de objeto no se debe manejar igual que una variable o un método. También definimos representaciones internas para tipos primitivos, objetos, genéricos, parámetros de tipo y funciones. Como la resolución inicial de TypePal no cubría todos nuestros casos, agregamos una pasada sobre el AST con las comprobaciones semánticas del lenguaje.

Comprobaciones implementadas

• Inferencia y compatibilidad de expresiones aritméticas, lógicas y comparaciones.

• Comprobación de anotaciones de tipo y asignaciones entre tipos incompatibles.

• Declaraciones val y var; detección de reasignación de variables inmutables val.

• Resolución de tipos de objetos y parámetros genéricos; comprobación de la cantidad de argumentos genéricos.

• Herencia y subtipado, incluidos casos de covarianza y violaciones de compatibilidad.

• Detección de supertipos desconocidos y ciclos de herencia.

• Comprobación de métodos inexistentes, argumentos de métodos y firmas incompatibles en sobrescrituras.

• Verificación de existencia de tipos de constructores y de tipos de sus argumentos.

Uno de los casos que usamos para probar el subtipado fue List[Animal] y List[Dog]. Cuando una variable espera List[Dog] y se intenta asignar List[Animal], el Checker reporta “Type mismatch: expected List[Dog] but found List[Animal]”. El mensaje

conserva el archivo y la línea. En esta versión el subrayado puede quedar al final de la expresión, aunque el error sí corresponde a la asignación.

Diagnósticos en VS Code

Después conectamos el Checker con ObjectlangLanguageServer.rsc. Hasta ese punto el servidor tenía parsing; agregamos analysis para que también pudiera mostrar los errores de tipos. La función analyzeObjectlang ejecuta checkObjectlang y convierte cada Message en un par <ubicación, mensaje> de la relación messages del Summary. Tuvimos un problema porque VS Code seguía reportando una ambigüedad en object Animal. Lo corregimos configurando el parser del servidor con allowAmbiguity=true y resolveLayoutAmbiguity, igual que en Parser.rsc. Al volver a registrar el lenguaje comprobamos que solo quedaba el error semántico esperado en la línea 6 de 06_covariance_violation.ol.

Pruebas y evidencia de validación

Para comprobar lo anterior creamos CheckerTests.rsc y organizamos ejemplos en examples/types/ok y examples/types/err. En total quedaron 21 pruebas: siete con programas válidos y catorce con errores intencionales. Para los válidos verificamos que la lista de mensajes estuviera vacía; para los inválidos, que se reportara al menos un error. Adicionalmente ejecutamos varios ejemplos directamente para revisar los mensajes que nos devolvía el Checker, ya que obtener un error no implica necesariamente que sea el correcto.

Batería automatizada completa

```
import CheckerTests;
:test
```

Resultado observado: Test report for CheckerTests — all 21/21 tests succeeded; bool: true

Casos válidos comprobados

```
import Checker;
import Parser;
import ObjectlangAST;
checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/ok/03_inference_and_operations.ol|)).messages;
checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/ok/06_method_assignment.ol|)).messages;
checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/run/snippet3.ol|)).messages;
```

Proyecto 3
Elementos Esenciales de Lenguajes de Programación
Andrés Huertas – 202420560
Juan David Colorado - 202415548

Resultado observado para los tres ejemplos: list[Message]: [].

Casos inválidos comprobados

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/05_annotation_mismatch.ol|)).messages;

Resultado observado: Type mismatch: expected Int but found String

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/09_unknown_method_call.ol|)).messages;

Resultado observado: Undefined method `fly` in object `Cat`

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/06_covariance_violation.ol|)).messages;

Resultado observado: Type mismatch: expected List[Dog] but found List[Animal]

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/12_immutable_reassignment.ol|)).messages;

Resultado observado: Cannot reassign immutable variable `x` declared with val

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/13_inheritance_cycle.ol|)).messages;

Resultado observado: Inheritance cycle involving `A` y `B` (dos mensajes)

checkObjectlang(parseObjectlang(|project://objectlang-project3/examples/types/err/14_override_signature.ol|)).messages;

Resultado observado: Override `speak` changes parameter type 0

Prueba directa del servidor de lenguaje

```
import ObjectlangLanguageServer;
import util::LanguageServer;
analyzeObjectlang(|project://objectlang-project3/examples/types/err/06_covariance_violation.ol|,
  parseObjectlang(|project://objectlang-project3/examples/types/err/06_covariance_violation.ol|)).messages;
```

Resultado observado: una relación rel[loc,Message] con el mensaje de incompatibilidad List[Dog] / List[Animal] y su ubicación. Tras registrar el servidor y recargar VS Code, la advertencia de ambigüedad desapareció y permaneció el diagnóstico de la línea 6.

Alcance y limitaciones conocidas

Las 21 pruebas pasaron, pero esto no significa que hayamos cubierto todas las combinaciones posibles del lenguaje. Por ejemplo, no comprobamos de forma exhaustiva los genéricos heredados, todos los patrones de los ciclos for ni las

asignaciones a estructuras anidadas. También hay una diferencia que identificamos al revisar el código: el Checker declara print y println con retorno Void, mientras que el intérprete devuelve stringV(text). Lo dejamos registrado porque habría que unificar ese comportamiento si se amplía el sistema de tipos. Finalmente, los tests de ejemplos inválidos verifican que exista al menos un diagnóstico; por eso revisamos manualmente los mensajes de los seis casos anteriores.